

Micro Frontend E-Commerce Platform — Master Implementation Prompt

Project Overview

You are a senior full-stack architect. Your task is to design and implement a **production-grade, micro frontend e-commerce platform** following **Clean Code Principles, DRY, and SOLID Principles** throughout every layer of the system.

Architecture: Micro Frontend Strategy

Use **Module Federation** (via `@module-federation/nextjs-mf` or Webpack 5 Module Federation) to compose independent MFE apps into a unified shell.

MFE Decomposition

MFE App	Responsibility	Port
<code>shell</code>	App shell, routing, global layout, auth guard	3000
<code>mfe-auth</code>	Login, Register, OAuth, Email verify, Password reset	3001
<code>mfe-catalog</code>	Product listing, detail, search, categories, reviews	3002
<code>mfe-cart</code>	Cart, guest cart, cart summary	3003
<code>mfe-checkout</code>	Checkout flow, order summary, payment	3004
<code>mfe-orders</code>	Order history, tracking, cancellation	3005
<code>mfe-profile</code>	User profile, addresses, loyalty points	3006
<code>mfe-promotions</code>	Coupons, flash sales, bundles	3007
<code>mfe-notifications</code>	In-app, push, email notification UI	3008
<code>mfe-admin</code>	Admin dashboard, analytics, inventory	3009

MFE Communication Contract

- Shared state:** Zustand store exposed via Module Federation `shared` config

- **Events:** Custom event bus ((`EventEmitter`)) for cross-MFE communication
 - **Auth token:** Stored in (`httpOnly`) cookie; read via (`/api/auth/me`) in each MFE
 - **Shared UI:** Design system published as (`@myapp/ui`) (internal npm package or shared remote)
-

Frontend Tech Stack

```
Framework:      Next.js 15 (App Router, Turbopack)
Language:       TypeScript 5.x (strict mode)
MFE:            Webpack 5 Module Federation (@module-federation/nextjs-mf)
State:          Zustand (global) + TanStack Query v5 (server state)
Forms:          React Hook Form + Zod
Styling:         Tailwind CSS v4 + shadcn/ui + Radix UI primitives
Animation:       Framer Motion
Icons:           Lucide React
i18n:            next-intl
SEO:             Next.js built-in metadata API + next-sitemap
Image opt:       Next.js <Image> + Cloudinary
Analytics:       Vercel Analytics + PostHog
Testing:         Vitest + React Testing Library + Playwright (E2E)
Linting:         ESLint (Airbnb config) + Prettier + Husky + lint-staged
```

Backend Tech Stack

```
Framework:      NestJS 10 (modular monolith → ready for microservices)
Language:       TypeScript 5.x (strict mode)
API:            REST (Fastify adapter) + GraphQL (Apollo Federation, optional)
Auth:           Custom Guards + JWT (jose) + OAuth2 (Google, Facebook)
ORM:            Prisma 5 + PostgreSQL (primary DB)
Cache:          Redis (ioredis) – sessions, rate limiting, cart, queues
Queue:          BullMQ (email, notifications, order processing)
Search:         Elasticsearch or MeiliSearch (product search)
Storage:        AWS S3 / Cloudflare R2 (images, invoices)
Email:          Nodemailer + React Email templates
SMS:            Twilio (optional)
Push:           Firebase Cloud Messaging (FCM)
Payment:        Stripe + PayPal + VNPay + MoMo adapters
Shipping:       GHN + GHTK adapters (strategy pattern)
```

PDF:	Puppeteer or @react-pdf/renderer (invoices)
Validation:	class-validator + class-transformer (all DTOs)
Testing:	Jest + Supertest + Testcontainers
API Docs:	Swagger (OpenAPI 3.1) auto-generated
Logging:	Winston + Pino + ELK stack
Monitoring:	Prometheus + Grafana

Infrastructure

Containerisation:	Docker + Docker Compose (dev) → Kubernetes (prod)
CI/CD:	GitHub Actions
CDN:	Cloudflare
DNS/SSL:	Cloudflare / Let's Encrypt
Secret management:	HashiCorp Vault / AWS Secrets Manager
Database:	PostgreSQL 16 (primary) + Redis 7 (cache/queue)
Search:	Elasticsearch 8 / MeiliSearch
Object storage:	AWS S3 / Cloudflare R2
Monitoring:	Grafana + Prometheus + Sentry

Code Quality Mandates

Clean Code Principles

- All functions do **one thing** only (Single Responsibility at function level)
- Functions are ≤ 20 lines; files are ≤ 300 lines
- No magic numbers or strings — use named constants or enums
- Self-documenting names: no abbreviations, no `data`, `info`, `temp` variables
- Every public function/class has a JSDoc comment
- No commented-out code in commits

DRY Principles

- Extract all repeated logic into shared utilities (`@myapp/shared` package)
- Use generic base classes/hooks for CRUD patterns
- Shared Zod schemas between frontend and backend via a `@myapp/contracts` package

- Central error handling: one `GlobalExceptionHandler` (NestJS), one `ErrorBoundary` (React)
- Reusable UI: every UI element > used in 2 places → moves to `@myapp/ui`

SOLID Principles

- **S**: Each NestJS service handles one domain; each React component renders one concern
 - **O**: Use Strategy pattern for payment/shipping providers (open for extension, closed for modification)
 - **L**: All provider implementations are substitutable (e.g., `StripePaymentProvider` and `PayPalPaymentProvider` are interchangeable via `IPaymentProvider`)
 - **I**: Split large interfaces; no component or service is forced to depend on methods it doesn't use
 - **D**: Inject dependencies via NestJS DI container; no `new SomeService()` in business logic
-

Repository Structure

The project is split into **three separate Git repositories**:

```
GitHub Organisation: @myapp
├─ myapp-frontend      # Next.js MFE monorepo
├─ myapp-backend       # NestJS API monorepo
└─ myapp-contracts     # Shared types & Zod schemas (published to npm / GitHub Packages)
```

`myapp-contracts` (shared — published first)

Framework-agnostic. Both repos install this as a versioned npm package.

Published to GitHub Packages or a private npm registry.

Bumped with semantic versioning; both repos pin to a specific version.

```
myapp-contracts/
├─ src/
│   ├─ auth/
│   │   ├─ register.schema.ts
│   │   ├─ login.schema.ts
│   │   └─ index.ts
│   ├─ products/
│   ├─ cart/
│   ├─ orders/
│   ├─ payments/
│   ├─ reviews/
│   ├─ promotions/
│   └─ notifications/
└─ index.ts                # barrel export
├─ package.json            # name: "@myapp/contracts"
├─ tsconfig.json
└─ README.md
```

Usage in both repos:

```
bash
```

```
# frontend & backend both install the same package
pnpm add @myapp/contracts
```

myapp-frontend (Next.js MFE monorepo)

```
myapp-frontend/
├─ apps/
│   ├─ shell/                # MFE host app (port 3000)
│   ├─ mfe-auth/             # Auth pages (port 3001)
│   ├─ mfe-catalog/          # Product listing & detail (port 3002)
│   ├─ mfe-cart/             # Cart (port 3003)
│   ├─ mfe-checkout/         # Checkout flow (port 3004)
│   ├─ mfe-orders/           # Order history & tracking (port 3005)
│   ├─ mfe-profile/          # User profile (port 3006)
│   ├─ mfe-promotions/       # Coupons & flash sales UI (port 3007)
│   ├─ mfe-notifications/    # Notification feed (port 3008)
│   └─ mfe-admin/            # Admin dashboard (port 3009)
├─ packages/
│   └─ ui/                   # Shared design system (shadcn/ui base)
```

```

|   ├── shared/                # Frontend utilities (formatters, hooks, helpers)
|   ├── event-bus/             # Cross-MFE event bus
|   └── config/                # Shared ESLint, TS, Tailwind configs
└── .github/
    └── workflows/
        ├── ci.yml             # Lint + test + build all MFEs
        └── deploy.yml         # Deploy changed MFEs only (Turborepo affected)
└── .env.example               # NEXT_PUBLIC_API_URL, NEXT_PUBLIC_MFE_* ports
└── turbo.json
└── pnpm-workspace.yaml
└── docker-compose.yml         # Local dev: all MFEs + nginx reverse proxy

```

Key env vars:

env

```

NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_SHELL_URL=http://localhost:3000
NEXT_PUBLIC_MFE_AUTH_URL=http://localhost:3001
NEXT_PUBLIC_MFE_CATALOG_URL=http://localhost:3002
# ... etc

```

myapp-backend (NestJS API monorepo)

```

myapp-backend/
└── apps/
    └── api/                    # Main NestJS application (port 4000)
        ├── src/
        │   ├── modules/
        │   │   └── auth/
        │   │       ├── auth.module.ts
        │   │       ├── auth.controller.ts
        │   │       ├── auth.service.ts
        │   │       └── guards/
        │   │           ├── jwt-auth.guard.ts
        │   │           ├── roles.guard.ts
        │   │           ├── google-oauth.guard.ts
        │   │           └── facebook-oauth.guard.ts
        │   │       └── strategies/
        │   └── users/
        └── products/

```

```

├── categories/
├── cart/
├── orders/
├── payments/
├── shipping/
├── reviews/
├── promotions/
├── notifications/
├── admin/
├── common/
├── decorators/
├── filters/
├── guards/
├── interceptors/
├── pipes/
├── utils/
├── config/
├── main.ts
├── test/
├── packages/
│   ├── database/                # Prisma schema, migrations, seed scripts
│   │   ├── prisma/
│   │   │   ├── schema.prisma
│   │   │   ├── migrations/
│   │   │   └── seed.ts
│   │   └── package.json        # name: "@myapp/database"
│   └── .github/
│       ├── workflows/
│       │   ├── ci.yml          # Lint + test + build
│       │   └── deploy.yml      # Deploy to staging / production
│       └── .env.example
├── turbo.json
├── pnpm-workspace.yaml
├── docker-compose.yml          # Local dev: api + postgres + redis +
                                elasticsearch

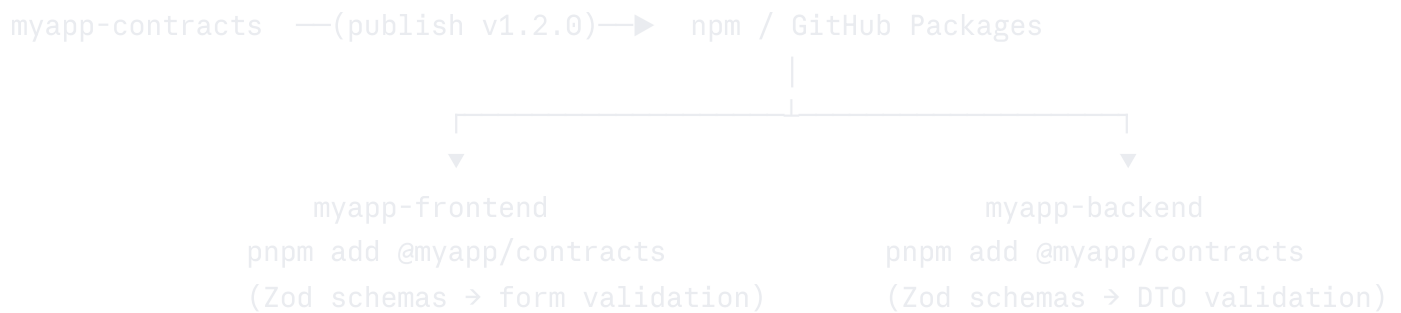
```

Key env vars:

env

```
DATABASE_URL=postgresql://user:pass@localhost:5432/myapp
REDIS_URL=redis://localhost:6379
JWT_SECRET=your-min-32-char-secret
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
FACEBOOK_CLIENT_ID=...
FACEBOOK_CLIENT_SECRET=...
FRONTEND_URL=http://localhost:3000
```

Cross-Repo Contract Flow



Versioning rule: Any breaking change to a contract schema requires a **major version bump** and coordinated upgrade PRs in both repos before deployment.

Local Development (all three repos)

```
bash
```



```
# 1. Start backend
cd myapp-backend
cp .env.example .env
docker-compose up -d          # postgres, redis, elasticsearch
pnpm install
pnpm dev                      # api runs on :4000

# 2. Start frontend
cd myapp-frontend
cp .env.example .env
pnpm install
pnpm dev                      # all MFEs run on :3000-:3009
```

Feature Implementation Specifications

1. User Management ((mfe-auth) + (api/modules/auth))

Frontend:

- Pages: (/login, /register, /verify-email, /forgot-password, /reset-password)
- Use (react-hook-form) + (zod) for all forms
- OAuth buttons trigger redirect to (/api/auth/google) and (/api/auth/facebook)
- After login, emit (AUTH_LOGIN) event on the event bus so all MFEs can react
- (useAuth()) hook is the single interface to auth state across all MFEs

Backend:

- (AuthModule) with (AuthService), (AuthController), (JwtAuthGuard), (RolesGuard), (GoogleOAuthGuard), (FacebookOAuthGuard)
- (JwtAuthGuard) — custom NestJS (CanActivate) implementation using (jose)'s (jwtVerify); extracts Bearer token from (Authorization) header or (auth-token) httpOnly cookie
- (RolesGuard) — reads (@Roles()) decorator metadata and checks against the (role) claim in the verified JWT payload
- (GoogleOAuthGuard) / (FacebookOAuthGuard) — custom guards that handle OAuth2 redirect + callback flow using the provider's SDK directly (no Passport); exchange auth code for tokens, fetch user profile, upsert user in DB

- `POST /auth/register` → hash password (bcrypt, rounds=12) → send verification email via BullMQ job → return `201`
- `POST /auth/login` → validate credentials → issue `accessToken` (15min) + `refreshToken` (7d, httpOnly cookie)
- `POST /auth/refresh` → verify refresh token → issue new access token
- `POST /auth/logout` → blacklist refresh token in Redis → clear cookie
- `GET /auth/verify-email?token=` → mark user as verified
- `POST /auth/forgot-password` → rate-limited (5 req/hour/IP) → send reset link via email
- `POST /auth/reset-password` → validate token (Redis TTL=1h) → update hash → invalidate all sessions

Interfaces (SOLID — I):

typescript

```
interface IAuthService {
  register(dto: RegisterDto): Promise<UserResponseDto>;
  login(dto: LoginDto): Promise<TokensDto>;
  logout(userId: string): Promise<void>;
  refreshTokens(refreshToken: string): Promise<TokensDto>;
  verifyEmail(token: string): Promise<void>;
  forgotPassword(email: string): Promise<void>;
  resetPassword(dto: ResetPasswordDto): Promise<void>;
}
```

2. Product Management (`mfe-catalog` + `api/modules/products`)

Frontend:

- `/products` — grid/list toggle, infinite scroll or paginated, filter sidebar (category, price, rating, tags), sort dropdown
- `/products/[slug]` — image gallery (Swiper), tabs (description, specs, reviews), related products
- Product search with debounce (300ms), powered by MeiliSearch instant-search
- Use TanStack Query for all product fetches with stale-while-revalidate

Backend:

- `ProductsModule`, `CategoriesModule`, `TagsModule`
 - `GET /products` — cursor-based pagination, filter by `categoryId`, `minPrice`, `maxPrice`, `tags[]`, `inStock`; sort by `price`, `rating`, `createdAt`
 - `GET /products/:slug` — include category, tags, first 5 reviews, stock level
 - `POST /products` (Admin) — multipart upload → S3 → store URLs
 - Full-text search delegated to MeiliSearch; Prisma used only for relational CRUD
 - Stock management: use Prisma transactions to prevent race conditions on purchase
-

3. Shopping Cart (`mfe-cart` + `api/modules/cart`)

Frontend:

- Cart state in Zustand; persisted to Redis (logged-in) or `localStorage` (guest)
- On login, merge guest cart with user cart (call `POST /cart/merge`)
- Cart drawer component (slide-in panel) consumed from `@myapp/ui`
- Emit `CART_UPDATED` event when cart changes so header badge updates

Backend:

- `CartModule` with Redis-backed cart storage (TTL = 30 days for guests, 90 days for users)
 - `GET /cart` → return cart with computed subtotal, tax, discounts
 - `POST /cart/items` → add item (validate stock availability)
 - `PATCH /cart/items/:id` → update quantity
 - `DELETE /cart/items/:id` → remove item
 - `POST /cart/merge` → merge guest cart into user cart on login
 - Apply active promotions/coupons in cart computation layer
-

4. Order & Checkout (`mfe-checkout` + `mfe-orders` + `api/modules/orders`)

Frontend (`mfe-checkout`):

- Step wizard: `Address` → `Shipping` → `Payment` → `Confirmation`
- Persist step state in Zustand; resumable on page refresh
- Before payment, display full `OrderSummary` component

- Redirect to `/orders/:id` after successful payment

Frontend (`mfe-orders`):

- `/orders` — paginated order history, filterable by status
- `/orders/:id` — timeline UI (status stepper), tracking info, cancel/refund CTA

Backend:

- Multi-step checkout uses a `CheckoutSession` entity stored in Redis (TTL = 2h)
 - `POST /orders` → validate cart → reserve stock (Prisma transaction) → create order → queue payment
 - Order status machine: `PENDING → PROCESSING → SHIPPED → DELIVERED` or `CANCELLED`
 - Status updates trigger BullMQ jobs: `SendOrderConfirmationEmailJob`, `SendShippingUpdateEmailJob`
 - `POST /orders/:id/cancel` → check cancellable window → trigger refund via payment adapter → update stock
-

5. Payment (`api/modules/payments`)

Strategy Pattern (SOLID — O, L):

typescript

```
interface IPaymentProvider {
  createPaymentIntent(order: Order, amount: Money): Promise<PaymentIntent>;
  confirmPayment(intentId: string): Promise<PaymentResult>;
  refund(transactionId: string, amount: Money): Promise<RefundResult>;
  handleWebhook(payload: unknown, signature: string): Promise<WebhookEvent>;
}

class StripePaymentProvider implements IPaymentProvider { ... }
class PayPalPaymentProvider implements IPaymentProvider { ... }
class VNPayPaymentProvider implements IPaymentProvider { ... }
class MoMoPaymentProvider implements IPaymentProvider { ... }
class CODPaymentProvider implements IPaymentProvider { ... }
```

- `PaymentService` accepts `IPaymentProvider` via DI — never references concrete classes
- Webhook endpoints per provider: `POST /payments/webhook/stripe`, etc.

- All webhooks verify signatures before processing
 - Invoice PDF generated via BullMQ job after successful payment → uploaded to S3 → URL stored on order
-

6. Shipping & Delivery (`api/modules/shipping`)

Strategy Pattern:

typescript

```
interface IShippingProvider {  
  calculateFee(params: ShippingCalculationParams): Promise<Money>;  
  createShipment(order: Order): Promise<Shipment>;  
  trackShipment(trackingCode: string): Promise<TrackingInfo>;  
}  
  
class GHNShippingProvider implements IShippingProvider { ... }  
class GHTKShippingProvider implements IShippingProvider { ... }  
class StandardShippingProvider implements IShippingProvider { ... }
```

- `GET /shipping/options?addressId=&cartId=` → returns available providers with fees + ETAs
 - Shipping address stored in `UserAddress` entity (users can have multiple, one default)
-

7. Reviews & Ratings (`api/modules/reviews`)

- `POST /reviews` → guard: user must have `DELIVERED` order containing `productId` → attach `verifiedPurchase: true`
 - `GET /reviews?productId=&page=&sortBy=` — paginated, sortable by date/rating
 - Moderation: `status: PENDING | APPROVED | REJECTED` (Admin approves)
 - Aggregate product rating recalculated via DB trigger or BullMQ job on review state change
 - Abuse detection: rate limit 1 review per product per user
-

8. Promotions & Discounts (`mfe-promotions` + `api/modules/promotions`)

- `CouponEntity`: code, discountType (`PERCENT | FIXED | FREE_SHIPPING`), value,

minOrderValue, maxUses, expiresAt, usedCount

- `FlashSaleEntity`: linked products, discountPercent, startAt, endAt — Redis TTL used for expiry
 - `LoyaltyPointEntity`: earn on purchase (configurable rate), redeem at checkout
 - `POST /promotions/apply` → validate coupon against cart → return discount breakdown
 - Admin can create/pause/expire any promotion via `mfe-admin`
-

9. Notifications (`mfe-notifications`) + (`api/modules/notifications`)

- All notifications dispatched via BullMQ jobs (never synchronous in request path)
 - Email templates built with `React Email` + rendered server-side
 - Push via FCM: users can register device tokens at `POST /notifications/device-tokens`
 - In-app notification feed: `GET /notifications` (paginated, mark-as-read)
 - Notification preferences: users can opt-out per channel per event type
-

10. Security

- **Authentication:** JWT (`accessToken` in memory / Authorization header, `refreshToken` in httpOnly cookie)
 - **RBAC:** `@Roles(Role.ADMIN, Role.SELLER)` guard on all privileged endpoints; roles stored on JWT + verified against DB
 - **Input validation:** every DTO uses `class-validator`; all API inputs sanitised with `class-sanitizer`
 - **Rate limiting:** `@nestjs/throttler` — global 100 req/min + tighter limits on auth endpoints
 - **CORS:** whitelist only known MFE origins
 - **Helmet:** all security headers set via `@nestjs/helmet`
 - **SQL injection:** impossible — Prisma uses parameterised queries
 - **Secrets:** never in code — env vars only, loaded via `@nestjs/config` + Joi validation on startup
 - **HTTPS:** enforced at infrastructure level (Cloudflare / Nginx)
-

11. Admin Dashboard (`mfe-admin`)

- Lazy-loaded; only accessible to `Role.ADMIN`
 - `/admin/products` — CRUD with bulk actions, image upload
 - `/admin/orders` — filter by status, date range; export CSV
 - `/admin/users` — ban/unban, role management
 - `/admin/promotions` — create/manage coupons, flash sales
 - `/admin/analytics` — revenue chart (Recharts), top products, order funnel, inventory alerts
 - Real-time order feed via WebSocket (`@nestjs/websockets`) + Socket.io
-

12. SEO & Performance

Frontend:

- Use Next.js `generateMetadata()` for dynamic OG/Twitter tags on product pages
- `next-sitemap` generates `/sitemap.xml` at build time
- All images use `next/image` with `priority` on above-fold, `lazy` elsewhere
- Route-level code splitting is automatic; MFEs load on demand
- Core Web Vitals budget: LCP < 2.5s, CLS < 0.1, INP < 200ms

Backend:

- Redis cache on: product listing (TTL=5min), product detail (TTL=10min), category tree (TTL=1h)
 - Cache invalidated on write via cache-aside pattern
 - DB query optimisation: all list endpoints use cursor pagination (no OFFSET); indexes on `slug`, `categoryId`, `status`, `createdAt`
 - Cloudflare CDN for static assets + image optimisation
-

Shared Contracts Package (`myapp-contracts` repo)

All DTOs, response shapes, and Zod schemas shared between frontend and backend. Installed in both repos as `@myapp/contracts` from GitHub Packages or a private registry:

typescript

```
// src/auth/register.schema.ts
export const registerSchema = z.object({
  name: z.string().min(2).max(100),
  email: z.string().email(),
  password: z.string().min(8).regex(/[A-Z]/).regex(/[0-9]/),
});

export type RegisterDto = z.infer<typeof registerSchema>;

export interface UserResponse {
  id: string;
  name: string;
  email: string;
  role: Role;
  isVerified: boolean;
  avatarUrl: string | null;
  createdAt: string;
}
```

Import `registerSchema` in both `mfe-auth` (form validation) and `api/modules/auth` (DTO validation) — zero duplication.

Testing Strategy

Layer	Tool	Coverage Target
Unit (services, hooks, utils)	Vitest / Jest	≥ 80%
Integration (API routes)	Supertest + Testcontainers	≥ 70%
Component	React Testing Library	≥ 70%
E2E (critical flows)	Playwright	All happy paths

Critical paths requiring E2E coverage:

1. Register → verify email → login
2. Browse products → add to cart → checkout → payment → order confirmation
3. Apply coupon → verify discount → place order

4. Admin: create product → list appears on catalog MFE
-

Monorepo Tooling

```
bash

# Package manager
pnpm workspaces

# Build system
Turborepo — parallel builds with remote caching

# turbo.json pipelines
build:  depends on ^build
test:   depends on build
lint:   no deps
dev:    no deps, persistent
```

Implementation Order

Follow this sequence to always have a runnable system:

1. **Monorepo scaffold** — Turborepo + pnpm workspaces + shared configs
2. `packages/contracts` — all shared Zod schemas and TypeScript types
3. `packages/ui` — design system (Button, Input, Card, Badge, Dialog, Toast)
4. `api` — Core — NestJS bootstrap, Prisma schema, global guards/filters/interceptors
5. `api/modules/auth` — full auth including OAuth + email queue
6. `shell` — Module Federation host, global layout, auth guard, event bus
7. `mfe-auth` — all auth pages
8. `api/modules/products` + `mfe-catalog` — product listing and detail
9. `api/modules/cart` + `mfe-cart` — cart with guest support
10. `api/modules/orders` + `mfe-checkout` + `mfe-orders` — full order flow
11. `api/modules/payments` — payment providers (Stripe first)
12. `api/modules/shipping` — shipping providers
13. `api/modules/reviews` + `api/modules/promotions` — reviews, coupons, flash sales

14. `api/modules/notifications` + `mfe-notifications` — notification system
 15. `mfe-admin` — admin dashboard
 16. `mfe-profile` + `mfe-promotions` — user profile and promotions UI
 17. **SEO, performance tuning, E2E tests**
 18. **Infrastructure** — Docker, K8s manifests, CI/CD pipelines
-

Implementation Workflow

Implement each task strictly one at a time. After completing each task, stop and follow this exact protocol before proceeding:

1. **Summarise** what was just implemented (files created/modified, patterns applied)
2. **Show a preview checklist** of what the user can verify:
 - Which URL/route to open in the browser
 - Which API endpoint to call (with example `curl` or HTTP snippet)
 - Which unit/integration test to run
 - What the expected visual or console output should be
3. **Ask explicitly:** "  Task [N] complete — please preview and confirm before I proceed to Task [N+1]."
4. **Wait for confirmation** — do not start the next task until the user replies with approval (e.g. "looks good", "proceed", "confirmed")
5. **If the user reports an issue**, fix it fully before moving on — never skip ahead

Preview Prompt Template (use after every task)

```
---
 Task [N] — [Task Name] — COMPLETE

 Files created/modified:
- path/to/file1.ts
- path/to/file2.tsx

 How to preview:
• Browser: http://localhost:[PORT]/[route]
• API test: curl -X POST http://localhost:3000/api/[endpoint] \
  -H "Content-Type: application/json" \
  -d '{ ... }'
```

- Run tests: `pnpm test --filter=[app-or-package]`
- Expected: [describe what the user should see or get back]

▶ Next task will be: Task [N+1] – [Next Task Name]

👉 Please preview the above and reply "proceed" when ready.

Rule: No task is considered done until the user has confirmed the preview. Quality over speed.

⚠ Non-Negotiable Rules

1. No **any** type — enable `strict: true` in all `tsconfig.json`
2. No **direct DB calls in controllers** — always go through the service layer
3. No **business logic in React components** — logic lives in hooks or services
4. **All async errors are caught** — use `asyncHandler` wrapper or NestJS exception filters
5. **Every API endpoint is documented** in Swagger before it ships
6. No **secrets in code** — `process.env` only, validated at startup with Joi
7. **Every PR requires** passing lint + unit tests + at least 1 integration test for new endpoints
8. **Database migrations** are never edited after they are committed — always create a new migration